

Sistemas de gestión empresarial

UD3 Modularidad. Listado de ejercicios 1

Resultado de aprendizaje (RA)	Criterios de evaluación (CE)	Calificación
RA6: Conoce el lenguaje de programación Python, su sintaxis, sus elementos básicos y sus funciones incorporadas más utilizadas como base para el desarrollo de módulos de sistemas ERP-CRM	6.3 Se ha trabajado con clases, objetos y funciones.	

Nota: Todos los ejercicios se harán utilizando funciones y en una carpeta con vuestro nombre y un apellido, cada ejercicio se debe hacer en un archivo .py con el nombre “ejercicio01.py”, “ejercicio02.py” y así sucesivamente.

1. Escriba un programa que pida un número. Después pregunte cuántos números se van a introducir, pida esos números, y escriba cuántos de esos números era mayor que el anterior.
2. Escriba un programa que pida tres números enteros distintos y que escriba una lista que empiece por el más pequeño y termine en el más grande.
3. Escriba un programa que pregunte cuántos números se van a introducir, pida esos números, y muestre un mensaje cada vez que un número no sea mayor que el anterior.
4. Escriba un programa que pida un número y dibuje dos cuadrados de ese número de estrellas en diagonal, como muestran los ejemplos siguientes:

```

Escriba un número entero positivo: 3
* * *
* * *
* * *

```

```

    * * *
    * * *
    * * *

```

```

Escriba un número entero positivo: 4
* * * *
* * * *
* * * *
* * * *

```

```

    * * * *
    * * * *
    * * * *
    * * * *

```

5. Escriba un programa que pida dos números (m y n) y que escriba n segmentos de m estrellas separados por m espacios, como muestran los ejemplos siguientes:

```

Escriba el tamaño del segmento: 4
Escriba el número de segmentos: 3
* * * *      * * * *      * * * *

```

6. Escribe una función filtrar_calorias (lista_comidas, max_calorias) que reciba una lista de tuplas (nombre_comida, calorías) y devuelva una nueva lista solo con las comidas cuya cantidad de calorías sea menor o igual que max_calorias. Si la lista está vacía, devuelve una lista vacía.



7. Implementa una función `modo_avion(texto, activar=True)` que reciba una cadena y un booleano.
 - Si `activar` es `True`, devuelve la cadena "[MODULO AVIÓN] " seguida del texto.
 - Si `activar` es `False`, devuelve solo el texto original.
8. Crea una función `maraton_series(horas)` que reciba un número de horas vistas de series y devuelva una tupla con (días, horas_restantes).
Ejemplo: 27 horas → (1, 3).
9. Define una función `mas_procrastina(diccionario_horas)` que reciba un diccionario con pares {nombre: horas_procrastinando} y devuelva el nombre de la persona que más horas ha procrastinado. Si el diccionario está vacío, devuelve `None`.
10. Implementa `generar_password(base="1234", repeticiones=3)` que devuelva una contraseña formada por la cadena `base` repetida el número de veces indicado y seguida de "!" al final.
Ejemplo: `generar_password("abc", 2)` → "abcab!".
11. Crea una función `estado_bateria(porcentaje)` que:
 - Devuelva "Perfecto" si porcentaje está entre 80 y 100.
 - Devuelva "Aceptable" entre 30 y 79.
 - Devuelva "Modo ahorro YA" entre 10 y 29.
 - Devuelva "Busca un enchufe" si es menor que 10.
12. Escribe una función `contador_likes(*likes_por_dia)` que reciba un número variable de enteros (likes de cada día) y devuelva: (Si no lo sabes, busca qué significa el asterisco como parte del parámetro de una función en Python)
 - El total de likes.
 - La media de likes.
 - El día con más likes (posición empezando en 1).
13. Crea una función `es_spam(asunto, *palabras_prohibidas)` que devuelva `True` si en el asunto aparece alguna de las palabras prohibidas (ignorando mayúsculas/minúsculas), y `False` en caso contrario.
14. Escribe una función `total_compra(lista_productos, iva=21)` donde `lista_productos` es una lista de precios (floats).
La función devuelve el total con IVA incluido.
Si la lista está vacía, devuelve 0.
15. Implementa `agregar_tarea (agenda, descripcion, prioridad=3)` donde:
 - `agenda` es una lista de diccionarios con claves 'descripcion' y 'prioridad'.
 - La función añade una nueva tarea con la prioridad indicada (1 muy alta, 5 muy baja) y devuelve la agenda ordenada de menor a mayor prioridad.
16. Define una función `contar_excusa(texto)` que reciba un texto y devuelva cuántas veces aparecen palabras típicas de excusa: 'perro', 'ordenador', 'luz', 'internet'.
No hace falta distinguir mayúsculas/minúsculas.
17. Escribe una función `normalizar_usuario(nombre)` que:
 - Elimine espacios al principio y final.
 - Reemplace los espacios intermedios por guiones bajos.
 - Convierta todo a minúsculas.
Ejemplo: ' Juan Pérez López ' → 'juan_pérez_lópez'.
18. Crea `karma_programador(commits, bugs, cafes=0)` que devuelva un 'índice de karma' calculado como:
$$\text{karma} = \text{commits} * 2 - \text{bugs} * 3 + \text{cafes}$$

El karma no puede ser menor que 0; si el resultado es negativo, devuelve 0.